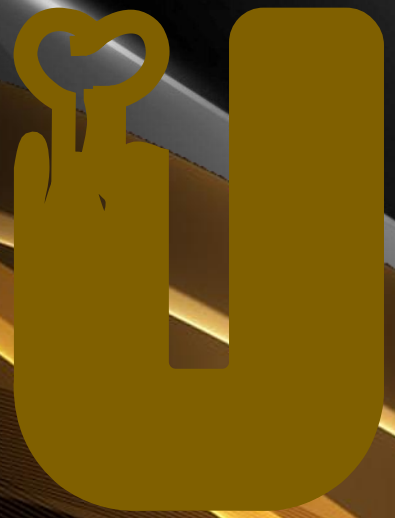


U5 – Pilas y Colas

Pilas & Colas

Stacks and Queues

STG I – U5 Pilas



Pilas| Stacks & Colas | Queue

CORE

Pilas - Stacks |
Pilas - Stacks | Operaciones Primitivas - Stacks
Pilas – Stacks | 1. Push
Pilas – Stacks | 2. Pop
Pilas – Stacks | 3. GetTop

TIPOS COLAS

Pilas – Stacks | ABSTRACT DATA TYPE
Pilas - Stacks | Representation of StackS using Sequential Organization (Arrays)
Pilas – Stacks | Create
Pilas – Stacks | Empty
Pilas – Stacks | GetTop
Pilas – Stacks | Push
Pilas – Stacks | Pop
Pilas – Stacks | Code Example

Pilas – Stacks | Using Template
Pilas – Stacks | Using STL
Pilas – Stacks | APPLICATIONS of stack

STG II – U5 Colas



Queue

CORE

Queue (Colas) | Introducción
Queue (Colas) | Create & getFront
Queue (Colas) | Is_Empty & Is_Full
Queue (Colas) | Add & Delete
Queue (Colas) | Example

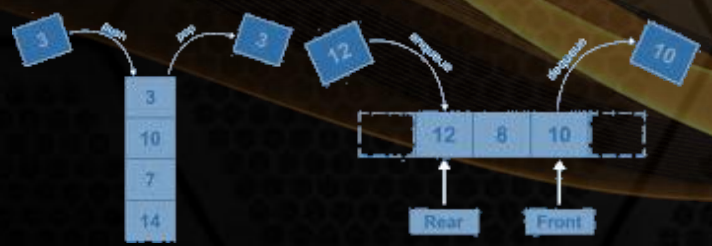
CIRCULARES PRIORITARIAS DEQUE

Queue (Colas) | Circular Queue
Queue (Colas) | Circular Queue Example

Queue (Colas) | Multiqueues
Queue (Colas) | Deque
Queue (Colas) | Priority Queue
Queue (Colas) | Array Implementation of Priority Queue

Queue (Colas) | Priority Queue
Queue / Deque in the Standard Template Library
Queue (Colas) | QUEUES USING TEMPLATE
Deque (Colas) | DEQUE USING TEMPLATE
Queue (Colas) | Resumen

Pilas y Colas| TDA
Data Structure - Stack & Queue .



Como se explicó en las Unidades pasadas, los *"**tipos de datos abstractos** **nos permiten retrasar la implementación específica de un tipo de datos hasta que se entienda bien qué operaciones se requieren para operar sobre los datos"***.
De hecho, estas operaciones determinan qué implementación del tipo de datos es más eficiente en una situación particular. Esta situación se ilustra con dos tipos de datos, ***pilas y colas***, que se describen mediante una lista de operaciones. Solo después de determinar la lista de las operaciones requeridas, presentamos algunas implementaciones posibles y las comparamos.

Pilas| Stacks & **Colas** | Queue

Las pilas y las colas son estructuras de datos especiales en las que se realizan operaciones de inserción y eliminación.
Estas se realizan en lugares específicos, en lugar de intermedios o cualquier otro aleatorio posiciones. Estos son casos especiales de listas ordenadas. Como hemos visto en la Unidad 3, las estructuras lineales de datos como matrices y listas enlazadas nos permiten insertar o eliminar un elemento desde cualquier posición en la lista; pilas y colas son listas lineales con restricciones en estos operaciones. Analicemos estos conceptos en detalle.

Las **pilas y las colas** son las dos estructuras de datos donde se realizan las operaciones de inserción y eliminación.
se aplica solo en fines específicos. Estos son casos especiales de listas ordenadas y también se les llama listas lineales controladas.
Existe una gran variedad de aplicaciones de software donde necesitamos estas operaciones restringidas de estructura de datos. Los siguientes son algunos ejemplos donde las pilas y las colas se utilizan generalmente:

1. Las colas se utilizan mucho en aplicaciones que mantienen una lista de trabajos de impresión en espera en un impresora de red Aquí, se mantiene una cola que puede contener todas las solicitudes de impresión de diferentes usuarios.

2. El manejo de llamadas de función en programas muy a menudo restringe el acceso en un extremo para realizar un seguimiento de la posición de regreso. En tal implementación, necesitamos usar pilas. Podemos hacer un seguimiento de
la dirección de retorno a la función anterior después de proporcionar/finalizar una llamada de función usando pilas.

Pilas | Stacks
Introducción.

Primitive Operation Stacks (The Basics)

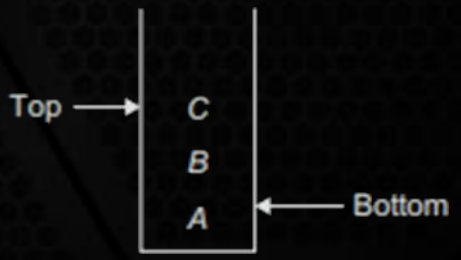
Pilas - Stacks

En nuestra vida cotidiana, nos encontramos con muchos ejemplos de pilas, por ejemplo, una pila de libros, una pila de platos o una pila de sillas. La pila de estructura de datos es muy similar a estos ejemplos prácticos



Consideremos una pila de libros sobre una mesa. Podemos poner fácilmente un nuevo libro en la parte superior de la pila, y de manera similar, podemos eliminar fácilmente el libro superior en comparación con los libros que se encuentran en el medio o en las posiciones inferiores. De la misma manera, solo se puede acceder al elemento superior de una pila, mientras que el acceso directo de otras posiciones intermedias no es factible. Los elementos se pueden agregar o quitar de un solo extremo, llamado la parte superior de una pila. Las estructuras de datos lineales como matrices y listas enlazadas permiten a los usuarios insertar o eliminar un elemento en cualquier posición de la lista, es decir, podemos insertar o eliminar un elemento al principio, al final, o en cualquier posición intermedia.

Una pila se define como una lista restringida donde todas las inserciones y eliminaciones se realizan solo en un extremo, **la parte superior**. Cada tipo de datos abstracto de pila (ADT) tiene un miembro de datos, comúnmente denominado superior, que apunta al elemento superior de la pila. **Hay dos operaciones básicas push y pop** que se pueden realizar en una pila; la inserción de un elemento en la pila se denomina push y la eliminación de un elemento de la pila se denomina pop. En las pilas, no podemos acceder a los elementos de datos desde ninguna posición intermedia que no sea la posición superior. Dada una pila S = (a1, a2, ..., an). Decimos que como a1 es el elemento más inferior, an está en la parte superior de la pila, y se dice que el elemento a1 + 1 está en la parte superior de a1 , $1 < i < n$. En la Fig. , S = (A, B, C), donde A es el elemento más inferior y C es el elemento superior



Pilas Operaciones Primitivas - Stacks

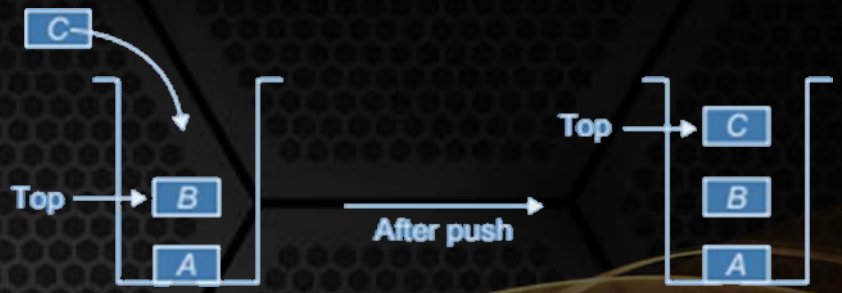
Las tres operaciones comunes de pila son: push, pop y getTop. Además de estos, hay algunas operaciones más que se pueden implementar en una pila, como la inicialización de stack_, la stack_empty y la stack_full. La operación stack_initialization prepara la pila para su uso y la establece en un estado vacante. La operación stack_empty simplemente prueba si la pila está vacía. La operación stack_empty es útil como protección contra un intento de sacar un elemento de una pila vacía. Hacer estallar una pila vacía es una condición de error. La condición de stack_empty también se denomina subflujo de pila. En condiciones ideales, las pilas deben poseer una capacidad infinita para que los elementos posteriores siempre puedan ser empujados, independientemente del número de elementos ya presentes en la pila. Sin embargo, las computadoras siempre tienen una capacidad de memoria finita, y necesitamos verificar la condición de stack_full antes de hacer push porque empujar un elemento en una pila completa también es una condición de error. Tal condición de pila completa se llama desbordamiento de pila (StackOverFlow). Otra operación de pila es GetTop. Esto devuelve el elemento superior de la pila sin realmente explotarlo. Algunas operaciones de pila más incluyen atravesar la pila, contar el número total de elementos en la pila y copiar la pila. Recordemos rápidamente todas las operaciones de la pila:

1. **Push**: inserta un elemento en la parte superior de la pila
2. **Pop** :permite eliminar un elemento de la parte superior de la pila
3. **GetTop**: lee (solo lee, no elimina) un elemento de la parte superior de la pila
4. **Stack_initialization**: permite configurar la pila en una condición vacía
5. Vacío (is**Empty**): permite comprobar si la pila está vacía
6. Lleno: comprueba si la pila está llena

La característica más importante de la pila es que el último elemento insertado en ella es el primero en suprimirse. Por esta razón, una pila se denomina como una estructura **LIFO (Last In First Out)** en la cual, el último elemento insertado, será el primero en ser eliminado.

Pilas – Stacks | 1. Push

La operación **push** inserta un elemento en la parte superior de la pila. El elemento agregado recientemente siempre está en la parte superior de la pila. Antes de cada empujón, debemos asegurarnos de si hay espacio para un nuevo elemento.



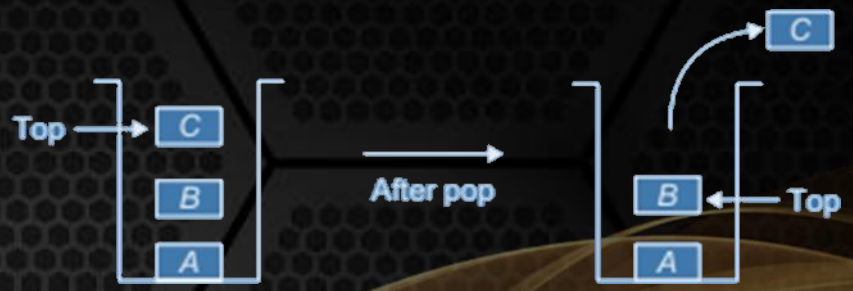
Cuando no hay espacio para acomodar el nuevo elemento en la pila, se dice que la pila está llena (ver Fig). Si la operación push se realiza cuando la pila está llena, se dice que está en estado de desbordamiento (stackoverflow), es decir, no se puede agregar ningún elemento cuando la pila está llena. La operación push modifica la parte superior ya que el elemento recién insertado se convierte en el elemento superior



Un stack overflow, en español **desbordamiento de pila**, es un error que ocurre cuando se llena el **stack** y por lo tanto no se pueden crear más subrutinas anidadas (imagina que la pila de libros llegó al techo y no hay forma de ubicar más libros sobre esa pila).

Pilas – Stacks | 2. Pop

La operación pop elimina un elemento de la parte superior de la pila y devuelve el mismo al usuario. Modifica la pila para que el siguiente elemento se convierta en el elemento superior



Cuando no hay ningún elemento disponible en la pila, se dice que la pila está vacía. Si pop se realiza cuando la pila está vacía, entonces se dice que la pila está en un estado de desbordamiento



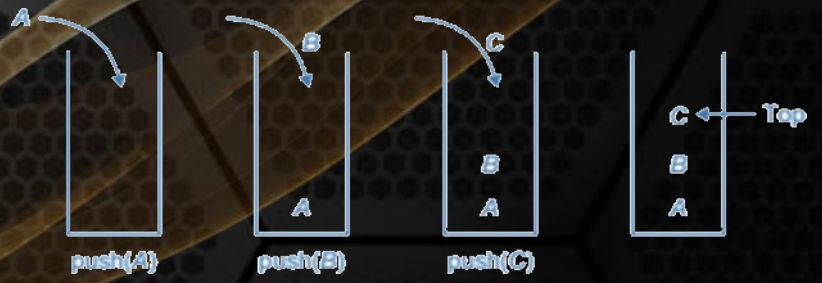
La operación pop no debe realizarse cuando la pila está vacía y, por lo tanto, antes de cada pop, debemos asegurarnos de que la pila no esté vacía. Después de eliminar el último elemento de la pila, la pila debe establecerse en un estado vacío

Pilas – Stacks | 3. GetTop

La operación getTop proporciona información sobre el elemento superior y devuelve el elemento en la parte superior de la pila. En esta operación, solo se devuelve una copia del elemento, que se encuentra en la parte superior de la pila. Por lo tanto, la parte superior sigue configurada en el mismo elemento



la suya es la diferencia clave entre las operaciones pop y getTop. La operación getTop no modifica la variable superior. Señala el error de desbordamiento de la pila si la pila está vacía. Como las operaciones de inserción y eliminación solo se permiten en un extremo de la pila, recupera los datos en el orden inverso en el que se almacenan los datos. En la Fig. , sea S = {A, B, C}, se observa como se llena.



Pilas | Stacks
In Action.

TDA

Stacks Abstract Data Type

Pilas – Stacks | ABSTRACT DATA TYPE

Veamos ahora el objeto de datos, las operaciones y los axiomas asociados con la pila. Cualquier conjunto de elementos que sean del mismo tipo de datos se puede utilizar como objeto de datos para pilas. El significado de **"mismo tipo de datos"** es que todos los elementos de la pila deben ser de la misma naturaleza, teniendo propiedades lógicas de representación comunes. Por ejemplo, pila de enteros, pila de nombres de estudiantes, pila de registros de empleados y pila de registros de procesos del sistema operativo.

Las cinco funciones siguientes comprenden una definición funcional de una pila:

1. **Create(S)**: crea una pila vacía
2. **Push(i, S)**: inserta el elemento i en la pila S y devuelve la pila modificada
3. **Pop(S)**: elimina el elemento superior de la pila S y devuelve la pila modificada
4 **GetTop(S)**: devuelve el elemento superior de la pila S
5. **Is_Empty(S)**: devuelve true si S está vacío y, de lo contrario, devuelve false

TDA

Pilas – Stacks | ABSTRACT DATA TYPE

Sin embargo, cuando elegimos representar una pila, debe ser posible construir estas operaciones. Antes de hacer esto, describamos formalmente la estructura de una pila.

ADT Stack (elements)

Funciones de la Pila	**Axiomas que describen la Pila.**	**Definen el Comportamiento LIFO**
1. Declare Create() -> stack 2. push(element, stack) -> stack 3. pop(stack) -> stack 4. getTop(stack) -> element 5. Is_Empty(stack) -> Boolean;	6. for all S stack, e element. 7. Is_Empty(Create) = true 8. Is_Empty(push(e, S)) = false 9. pop(Create()) = error	10. pop(push(e,S)) = S 11. getTop(Create) = error 12. getTop(push(e, S)) = e 13. end 14. end stack

Las cinco funciones con sus dominios y rangos se declaran en las líneas 1 a 5.
Las líneas 6 a 13 son el conjunto de axiomas que describen cómo se relacionan las funciones.
Las líneas 10 y 12 son importantes porque definen el comportamiento **LIFO** de la pila.
Esta descripción muestra una pila infinita sin límite superior o techo en el número de elementos especificados.
Para implementar la pila de ADT en C++, las operaciones a menudo se implementan como funciones para proporcionar abstracción de datos. Un programa que utiliza pilas accedería a las pilas solo a través de estas funciones y no se preocuparía por la implementación.

Pilas | Stacks
C++ Implementation.

Representation of StackS using Sequential Organization (Arrays)

Stacks | Representation of StackS using Sequential Organization (Arrays)

Una pila se puede implementar utilizando tanto una estructura de datos estática (array) como una estructura de datos dinámica (lista vinculada). La forma más sencilla de representar una pila es mediante el uso de un array unidimensional.

Una pila implementada mediante una array también se denomina pila contigua. Una array se utiliza para almacenar una lista ordenada de elementos. Una pila es una colección ordenada de elementos. Por lo tanto, sería muy simple administrar una pila cuando se representa utilizando un array.

La única dificultad con un array es su asignación de memoria estática. Una vez declarado, el tamaño no se puede modificar durante el tiempo de ejecución. Ya hemos leído que esto conduce a una mala utilización del espacio o a la incapacidad de acomodar todos los elementos de datos posibles. Esto se debe a que declaramos que una array es de tamaño máximo arbitrario antes de la compilación.



Stacks | Representation of StackS using Sequential Organization (Arrays)

Sea Stack[n] una array unidimensional. Cuando la pila se implementa utilizando Arrayas, uno de los dos lados de la array se puede considerar como el lado superior (superior) y el otro como el lado inferior (inferior) como en la Fig.



Discutamos el lado superior, que se usa más comúnmente. Los elementos se almacenan en la pila desde la primera ubicación en adelante. El primer elemento se almacena en la 0ª ubicación de la array Stack, lo que significa en Stack[0], el segundo elemento en Stack[1], el elemento ith en Stack[i - 1], y el enésimo elemento en Stack[n - 1]. Asociado con la array habrá una variable entera, top, que apunta al elemento superior de la pila. El valor inicial de top es -1 cuando la pila está vacía. Puede contener los elementos del índice 0 y puede crecer hasta un máximo de n - 1, ya que se trata de una pila estática que utiliza matrices. El código de programa proporciona la descripción de la clase Stack y enumera los prototipos de funciones para un conjunto de operaciones básicas

```
class Stack {

 private:

  int Stack[50];
  int MaxCapacity;
  int top;

 public:

  Stack() {

    MaxCapacity = 50;
    top = -1;
    currentsize = 0;
  }

    int getTop();
    int pop();
    void push(int Element);
    int Empty();
    int CurrSize();
    int IsFull();

};
```

Nota: La forma más sencilla de implementar una pila ADT es mediante Array. Inicializamos la variable Top a -1 usando un constructor para denotar una pila vacía. El elemento inferior se representa utilizando la 0ª posición, es decir, el primer elemento del array. El siguiente elemento se almacena en la 1ª posición y así sucesivamente. La variable TOP indica el elemento actual en la parte superior de la pila

Pilas – Stacks | 2. Create

La pila cuando se crea está inicialmente vacía. La implementación de la pila podría ser el uso de una array o el uso de una implementación de lista vinculada. Para la implementación de arreglos, su tamaño debe estar predefinido y su tiempo de implementación no debe exceder el tiempo de ejecución. Sin embargo, en caso de una implementación vinculada, esta limitación se supera.

Veamos primero una implementación de pila simple. Al final , estudiaremos otras mejores implementaciones basadas en Arrays que utilizan características de C++, como plantillas y Array dinámicos.

Para todas y cada una de las pilas, hay una variable de operador final operativa llamada TOP parte superior que apunta al elemento en la parte superior de la pila. Por lo tanto, esta variable entera contiene el índice de la array. También se puede implementar como una variable de puntero. Usémoslo actualmente como una variable entera. Aunque lo llamamos como un puntero que apunta al elemento superior de la pila, es una variable de índice entero.

```
class Stack
 {
  private:
     int Stack[50];
     int MaxCapacity;
     int top;
     public:
 Stack()
 {
     MaxCapacity = 50;
     top = -1;
 currentsize = 0;
 }
     int getTop();
     int pop();
     void push(int Element);
     int Empty();
     int CurrSize();
     int IsFull();
 };
```

Pilas – Stacks | 2. Create (cont.)

El constructor debe inicializar la parte superior de la pila, para representar una pila vacía, a un valor que represente la parte superior de la pila vacía. No podemos inicializarlo a uno de los valores en el rango de 0 a n - 1 porque estos son los índices de la array de pila. Los índices 0 a n - 1 representan una de las ubicaciones que van a contener los elementos de la pila. Sin embargo, se puede inicializar a cualquier valor entero arbitrario que no sea 0 a n - 1. Cada operación push aumenta la parte superior en una. Esto es para actualizar la parte superior para apuntar a un elemento recién agregado.

Cuando el elemento se agrega a la pila vacía, la parte superior debe establecerse en 0, ya que el nuevo elemento se almacenará en Stack[0].
Por lo tanto, es adecuado inicializar la parte superior a -1. Esta es la inicialización más adecuada en lugar de cualquier otro valor arbitrario. pila int[100]; int top = −1; Estas instrucciones crean una pila vacía de tamaño 100, que mantendrá valores enteros, y la parte superior de la variable se inicializa en -1.

```
class Stack
 {
  private:
     int Stack[50];
     int MaxCapacity;
     int top;
     public:
  Stack()
  {
     MaxCapacity = 50;
     top = -1;
     currentsize = 0;
  }
     int getTop();
     int pop();
     void push(int Element);
     int Empty();
     int CurrSize();
     int IsFull();
 };
```

Pilas – Stacks | Empty

Empty es una operación que toma la pila como argumento, comprueba si está vacía o no y devuelve el valor booleano true o false, respectivamente. El estado vacío de la pila se puede comprobar comparando el valor de top con el valor -1, porque top = -1 representa una pila vacía.

```
if(top == -1)
 return 1;
else
 return 0;
```

Pilas – Stacks | GetTop

La operación **getTop** comprueba el estado vacío de la pila. Si la pila está vacía, informa del mensaje de error 'stack underflow'; de lo contrario, devuelve una copia del elemento que está en la parte superior de la pila. Aquí, la parte superior no se actualiza ya que el elemento no se elimina de la pila; más bien, el elemento todavía está en la ubicación superior. El elemento se lee desde la pila. Por lo tanto, su comportamiento se puede describir utilizando la siguiente declaración:

```
if(top == -1)
 cout << "Stack underflow (empty)" << endl;
else
 return(Stack[top]);
```

Pilas – Stacks | Push

Empujar La operación **push** inserta un elemento en la pila de tamaño máximo MaxCapacity. La inserción de elementos solo es posible si la pila no está llena. No hemos discutido la operación completa en ADT. La pila está prácticamente llena cuando el tamaño de la array excede (o la memoria está llena, lo que puede suceder cuando usamos la representación de lista vinculada de la pila). Por lo tanto, el estado completo de la pila se puede verificar comparando la parte superior con MaxCapacity - 1. Si la pila no está llena, la parte superior se incrementa en 1 y el elemento se agrega en la parte superior de la pila. En resumen

```
if(top == MaxCapacity - 1)
 cout << "Stack overflow (full)";
else
{
 top ++; //increment top by one
 stack[top] = Element; //add the element in new top positio
n
}
```

Pilas – Stacks | **Pop**

La operación **pop** elimina el elemento en la parte superior de la pila y devuelve el mismo. Esto se hace solo si la pila no está vacía. Si la pila está vacía, no es posible eliminarla. Esto se comprueba mediante la función empty(). Si la pila no está vacía, entonces el elemento en la parte superior de la pila se devuelve y la parte superior se reduce en uno. Esto se ejecuta como

```
if(top == -1)
 cout << "Stack underflow\n";
else
 return(Stack[top--]);
```

La condición de pila llena indica que se necesita más almacenamiento, y en muchas aplicaciones de pilas, el estado vacío de la pila señala el final del procesamiento.

Pilas – Stacks | Code Example

El código de programa ilustra las operaciones básicas en una pila.

```cpp
#include <iostream>
using namespace std;

class Stack
{
private:
    int stack[50];
    int MaxCapacity;
    int top;

public:
    Stack()
    {
        MaxCapacity = 50;
        top = -1;
    }
    int getTop();
    int pop();
    void push(int Element);
    int Empty();
    int CurrSize();
    int IsFull();
};
```

```cpp
int Stack ::getTop()
{
    if (!Empty())
        return (stack[top]);
}
int Stack ::pop()
{
    if (!Empty())
        return (stack[top--]);
}
int Stack ::Empty()
{
    if (top == -1)
        return 1;
    else
        return 0;
}
int Stack ::IsFull()
{
    if (top == MaxCapacity - 1)
        return 1;
    else
        return 0;
}
```

```cpp
int Stack ::CurrSize()
{
    return (top + 1);
}
void Stack ::push(int Element)
{
    if (!IsFull())
        stack[++top] = Element;
}
int main()
{
    Stack S;
    S.pop();
    S.push(1);
    S.push(2);
    cout << S.getTop() << endl;
    cout << S.pop() << endl;
    cout << S.pop() << endl;
    return 0;
}
```

```
> .\Stack.exe
2
2
1
```



Stack.cpp

Pilas| Stacks

Stacks using Temlpate

Pilas – Stacks | Using Template

La pila que utiliza un Array y sus operaciones en el código de programa anterior se considera que funciona con datos enteros. Para definir el stack para floating point data, necesitamos cambiar int Stack[] a float Stack[] en la declaración de los miembros de datos de la clase. Esto se puede hacer cada vez que el tipo de datos de los elementos de la array varía, editando el código utilizando un editor de texto y luego recompilándolo…

En C++ una plantilla es una variable que se puede crear una instancia en cualquier tipo de datos. Este tipo de datos podría ser del tipo integrado o definido por el usuario.

```
template < class T >
  class Stack {

    private:
    T * Stack; // stack using pointer
    int top;
    int Size;

    public:
      Stack(int StackSize = 20); // constructor
    T & getTop();
    T & pop();
    void push(const T & Element);
    bool IsEmpty();
    int CurrSize();
  };

 template < class T >
   Stack < T > ::Stack(int StackSize): Size(StackSize) {
     Stack = new T[Size];
     top = -1;
   }
```

```
template < class T >
  T & Stack::getTop() {
    if !IsEmpty()
    return (Stack[top]);
    else
    cout << "Stack is Empty" << endl;
  }

template < class T >
  T & Stack::pop() {
    if !IsEmpty()
    return (Stack[top--]);
    else
      cout << "Stack is Empty" << endl;
  }

Bool Stack::IsEmpty() {
  if (top == -1)
    return 1;
  else
    return 0;
}

Bool Stack::IsFull() {
  if (top == MaxCapacity- 1)
    return 1;
  else
    return 0;
}

int Stack::CurrSize() {
  return (top + 1);
}

void Stack::push(const T & Element) {
  if (!IsFull())
    cout << "Stack is Full" << endl;
  else
    Stack[++top] = Element;
}
```

Pilas| Stacks

Inplementation in STL

Pilas – Stacks | Using STL

Las pilas son un tipo de adaptadores de contenedor con tipo de trabajo **LIFO (Last In First Out)**, donde se agrega un nuevo elemento en un extremo (superior) y se elimina un elemento de ese extremo solamente. Stack utiliza un objeto encapsulado de vector o deque (de forma predeterminada) o list (clase de contenedor secuencial) como su contenedor subyacente, proporcionando un conjunto específico de funciones miembro para acceder a sus elementos.

Sintaxis de pila:
Para crear una pila, debemos incluir el archivo de encabezado <stack> en nuestro código. A continuación, utilizamos esta sintaxis para definir el std::stack:

Las funciones asociadas con la pila son:
empty() – Devuelve si la pila está vacía – Complejidad de tiempo: O(1)
size() – Devuelve el tamaño de la pila – Complejidad de tiempo : O(1)
top() – Devuelve una referencia al elemento más alto de la pila – Complejidad de tiempo : O(1)
push(g) – Agrega el elemento 'g' en la parte superior de la pila – Complejidad de tiempo : O(1)
pop() – Elimina el elemento superior más importante de la pila – Complejidad de tiempo : O(1)

Pilas – Stacks | Using STL

Las pilas son un tipo de adaptadores de contenedor con tipo de trabajo LIFO (Last In First Out), donde se agrega un nuevo elemento en un extremo (superior) y se elimina un elemento de ese extremo solamente. Stack utiliza un objeto encapsulado de vector o deque (de forma predeterminada) o list (clase de contenedor secuencial) como su contenedor subyacente, proporcionando un conjunto específico de funciones miembro para acceder a sus elementos.

Sintaxis de pila:
Para crear una pila, debemos incluir el archivo de encabezado <stack> en nuestro código.

Las funciones asociadas con la pila son:

empty() – Devuelve si la pila está vacía – Complejidad de tiempo: O(1)
size() – Devuelve el tamaño de la pila – Complejidad de tiempo : O(1)
top() – Devuelve una referencia al elemento más alto de la pila – Complejidad de tiempo : O(1)
push(g) – Agrega el elemento 'g' en la parte superior de la pila – Complejidad de tiempo : O(1)
pop() – Elimina el elemento superior más importante de la pila – Complejidad de tiempo : O(1)

```cpp
#include <iostream>
#include <stack>
using namespace std;
int main() {
    stack<int> stack;
    stack.push(21);
    stack.push(22);
    stack.push(24);
    stack.push(25);

        stack.pop();
    stack.pop();

    while (!stack.empty()) {
        cout << stack.top() <<" ";
        stack.pop();
    }
}
```

```
❯ .\stack_STL.exe
22 21
```

stack_STL.cpp

Pilas – Stacks | APPLICATIONS of stack

La estructura de datos de pila se utiliza en una amplia gama de aplicaciones. Algunos de ellos son los siguientes:

1. Convertir la expresión infix en postfix y expresiones de prefijo
2. Evaluación de la expresión postfix
3. Comprobación de paréntesis bien formados (anidados)
4. Invertir una cadena
5. **Procesamiento de llamadas a la función (U6 - Recursividad)**
6. Análisis (análisis de la estructura) de programas informáticos
7. Simulación de recursión
8. En cálculos como la conversión decimal a binaria
9. En algoritmos de backtracking (a menudo utilizados en optimizaciones y en juegos)

Plase Comercial Here...|.

Cooming son...Queue

Stage II Colas| Introducción.

Queue

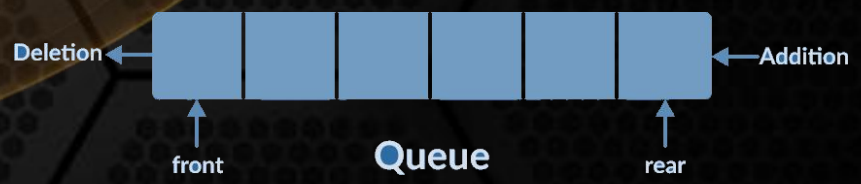
Colas | Introducción.

Queue

Queue (Colas) | Introducción

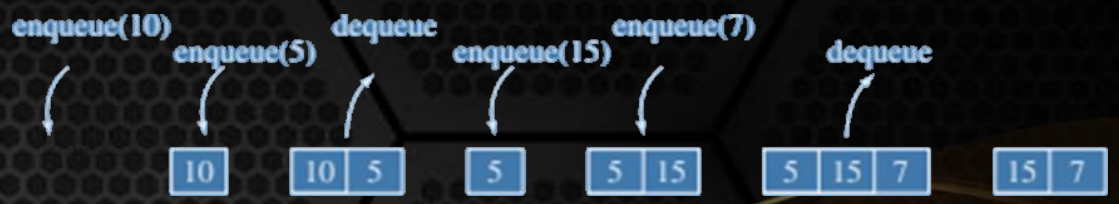
Una cola es simplemente una fila de espera que crece agregando elementos a su extremo y se encoge al tomar elementos de su frente.
A diferencia de una pila, una cola es una estructura en la que se utilizan ambos extremos: uno para agregar nuevos elementos y otro para eliminarlos.
Por lo tanto, el último elemento tiene que esperar hasta que se eliminen todos los elementos que le preceden en la cola.
Una cola es una **estructura FIFO**: primero en entrar/primero en salir.
Las operaciones de cola son similares a las operaciones de pila.
Las siguientes operaciones son necesarias para administrar correctamente una cola:

- **clear()**::Borrar la cola.
- **isEmpty()**:compruebe si la cola está vacía.
- **enqueue(el)**:permite colocar el elemento el al final de la cola.
- **dequeue()**:permite tomar el primer elemento de la cola.
- **firstEl()**: permite devolver el primer elemento de la cola sin quitarlo



Queue (Colas) | Introducción

la serie de operaciones de **queue y deque** se muestra en la figura. Esta vez, a diferencia de las pilas, los cambios deben supervisarse tanto al principio de la cola como al final. Los elementos se encuentran en un extremo y se desenredan del otro. Por ejemplo, después de poner en cola 10 y, a continuación, 5, la operación de descolado quita 10 de la cola



Para realizar una cola como un tipo de datos abstracto (ADT), necesitamos una estructura de datos adecuada para almacenar los elementos en la cola y las funciones que operan en ella.
Las operaciones básicas realizadas en la cola incluyen agregar y eliminar un elemento, atravesar la cola, verificar si la cola está llena o vacía y encontrar quién está en la parte delantera y quién está en los extremos traseros.
Un conjunto mínimo de operaciones en una cola es el siguiente:

1. create()—crea una cola vacía, Q
2. add(i,Q): agrega el elemento i al extremo posterior de la cola, Q y devuelve la nueva cola
3. delete(Q): saca un elemento del front-end de la cola y devuelve la cola resultante.
4. getFront(Q): devuelve el elemento que se encuentra en la posición delantera de la cola
5. Is_Empty(Q): devuelve true si la cola está vacía; De lo contrario, devuelve false

Queue (Colas) | Create & getFront

Create Esta operación debería crear una cola vacía. Aquí máximo es el tamaño inicial máximo que se define

```
#define max 50
int Queue[max];
int Front = Rear = -1;
```

Además de una cola de Array unidimensional, necesitamos dos variables más, (Front) Delantera y (Queue) Trasera. Esta declaración crea una cola vacía de tamaño máximo. Las dos variables Front y Rear se inicializan para representar una cola vacía. En general, es adecuado establecer Front en una posición detrás del frente real de la cola y establecer la parte trasera en el último elemento de la cola. Por lo tanto, la condición Adelante = Trasero indica una cola vacía. Como nuestro índice de array oscila entre 0 y (máx. − 1), la parte delantera y trasera se inicializan a -1

getFront La operación getFront devuelve el elemento en la parte delantera, pero a diferencia de delete, esto no actualiza el valor de Front.

```
int getFront()
{
 if(Is_Empty())
  cout << "Sorry, queue is Empty";
 else
  return(Queue[Front + 1]);
}
```

Queue (Colas) | Is_Empty & Is_Full

Is_Empty Esta operación comprueba si la cola está vacía o no. Esto se confirma comparando los valores de Delantero y Trasero. Si Adelante = Atrás, entonces Is_Empty devuelve true, else devuelve false.

```
bool Is_Empty()
{
 if(Front == Rear)
 return 1;
 else
 return 0;
}
```

Is_Full En la definición del ADT de cola, no se incluye la función para comprobar la condición de Queue_Full. Cuando entramos en una implementación de arreglo de discos, debido a su tamaño fijo, debemos verificar el estado de la cola para que esté lleno. Se recomienda que antes de eliminar un elemento de la cola, debemos comprobar si la cola está vacía o no. Del mismo modo, antes de la inserción, se debe comprobar la cola para el estado Queue_Full. Cuando la parte trasera apunta a la última ubicación de la array, indica que la cola está llena, es decir, que no hay espacio para acomodar más elementos.

```
bool Is_Full()
{
 if(Rear == max - 1)
 return 1;
 else
 return 0;
}
```

Queue (Colas) | Add & Delete

Add Esta operación agrega un elemento en la cola si no está lleno. Como La parte trasera apunta al último elemento de la cola, el nuevo elemento se agrega en la ubicación (trasera + 1).

```
void Add(int Element)
{
 if(Is_Full())
 cout << “Error, Queue is full”;
 else
 Queue[++Rear] = Element;
}
```

Delete Esta operación elimina un elemento del frente de la cola y establece Front para que apunte al siguiente elemento. El frente se puede inicializar a una posición menos que el frente real. Primero debemos incrementar el valor de Front y luego eliminar el elemento.

```
int Delete()
{
 if(Is_Empty())
 cout << “Sorry, queue is Empty”;
 else
 return(Queue[++Front]);
}
```

Queue (Colas) | Example

```
//Queue ADT
#include <iostream>
using namespace std;


class queue
{
private:

 int Rear, Front;
 int Queue[50];
 int max;
 int Size;

public:

 queue()
 {
 Size = 0; max = 50;
 Rear = Front - 1;
 }

 int Is_Empty();
 int Is_Full();
 void Add(int Element);
 int Delete();
 int getFront();
};
```

```
int queue :: Is_Empty()
{
 if(Front == Rear)
 return 1;
 else
 return 0;
}

int queue :: Is_Full()
{
 if(Rear == max - 1 )
 return 1;
 else
 return 0;
}

void queue :: Add(int Element)
{
 if(!Is_Full())
 Queue[++Rear] = Element;
 Size++;
}

int queue :: getFront()
{
 if(!Is_Empty())
 return(Queue[Front + 1]);
}
```

```
int queue :: Delete()
{
 if(!Is_Empty())
 {
 Size--;
 return(Queue[++Front]);
 }
}

int main()
{
 queue Q;
 Q.Add(11);
 Q.Add(12);
 Q.Add(13);
 cout << Q.Delete() << endl;
 Q.Add(14);
 cout << Q.Delete() << endl;
 cout << Q.Delete() << endl;
 cout << Q.Delete() << endl;
 cout << Q.Delete() << endl;
 Q.Add(15);
 Q.Add(16);
 cout << Q.Delete() << endl;

return 0;
}
```



Queue.cpp

Colas Circulares | Introducción.

Circle Queue

Queue (Colas) | Circular Queue

De la demostración de la ejecución de algunas operaciones push y pop se puede concluir que las colas lineales que utilizan arreglos tienen ciertos inconvenientes enumerados de la siguiente manera:

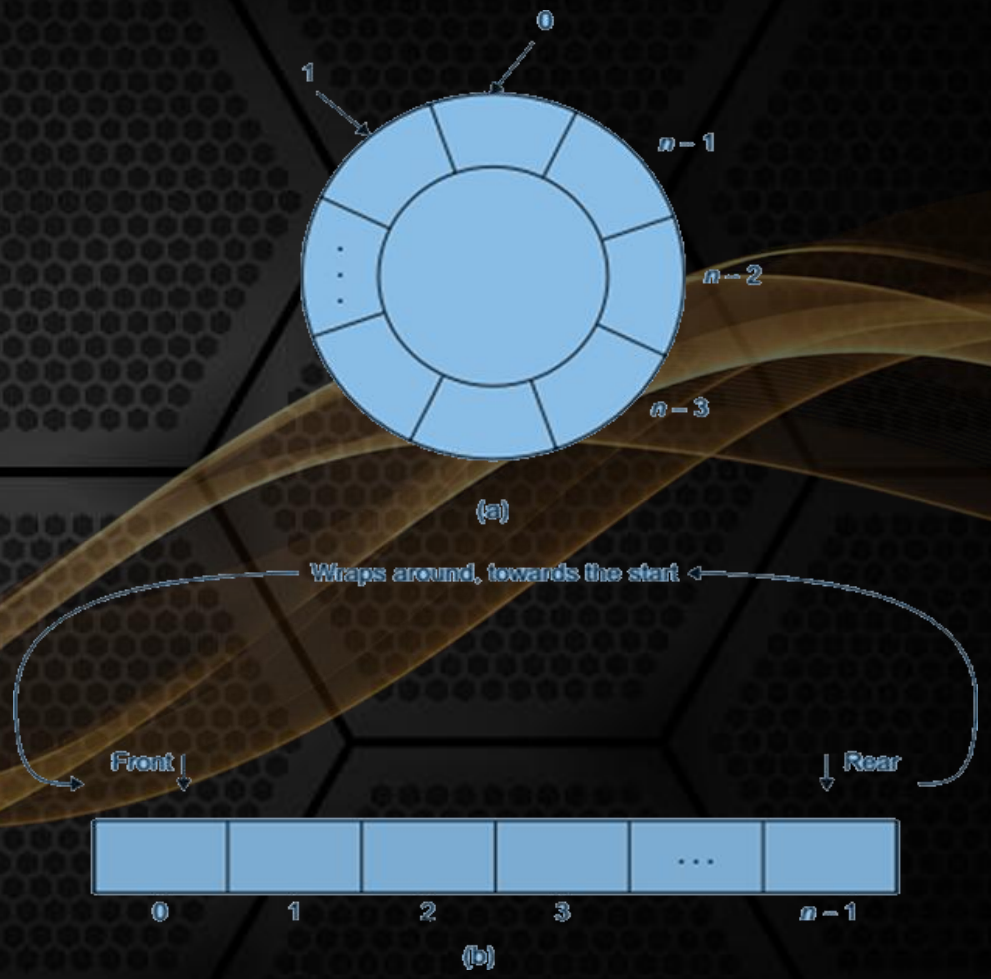
1. La cola lineal es de un tamaño fijo. Por lo tanto, el usuario no tiene la flexibilidad de cambiar dinámicamente el tamaño de la cola.
2. Un tamaño máximo de colas declarado arbitrariamente conduce a una mala utilización de la memoria. Por ejemplo, la cola se declara de tamaño 1000 y solo se utilizan 20 de ellos.
3. Necesitamos escribir un código adecuado para hacer que la parte delantera se ponga al día regularmente con la parte trasera y restablezca ambas. La implementación de Arrays de colas lineales conduce al estado Queue_Full aunque la cola no esté realmente llena.
4. Para evitar esto, cuando se señala Queue_Full, debemos rebobinar toda la cola a la ubicación de inicio original (si hay ubicaciones vacías) para que el primer elemento esté en la 0ª ubicación y front se establezca en -1. Tal movimiento de datos es una forma eficiente de evitar este inconveniente.

× La técnica que esencialmente permite que la cola se envuelva al llegar al final de la array elimina estos inconvenientes. Tal técnica que permite que las colas se envuelvan en cola de extremo a inicio se denomina **cola circular**. Virtualmente, queremos que el proceso de inserción y la parte trasera envuelvan la cola.

Queue (Colas) | Circular Queue

Por lo tanto, se obtiene una representación de cola más eficiente implementando el array Q como circular. Aquí, a medida que continuamos agregando elementos a la cola y llegamos al final de la array, el siguiente elemento se almacena en la primera ranura de la array si está vacía.

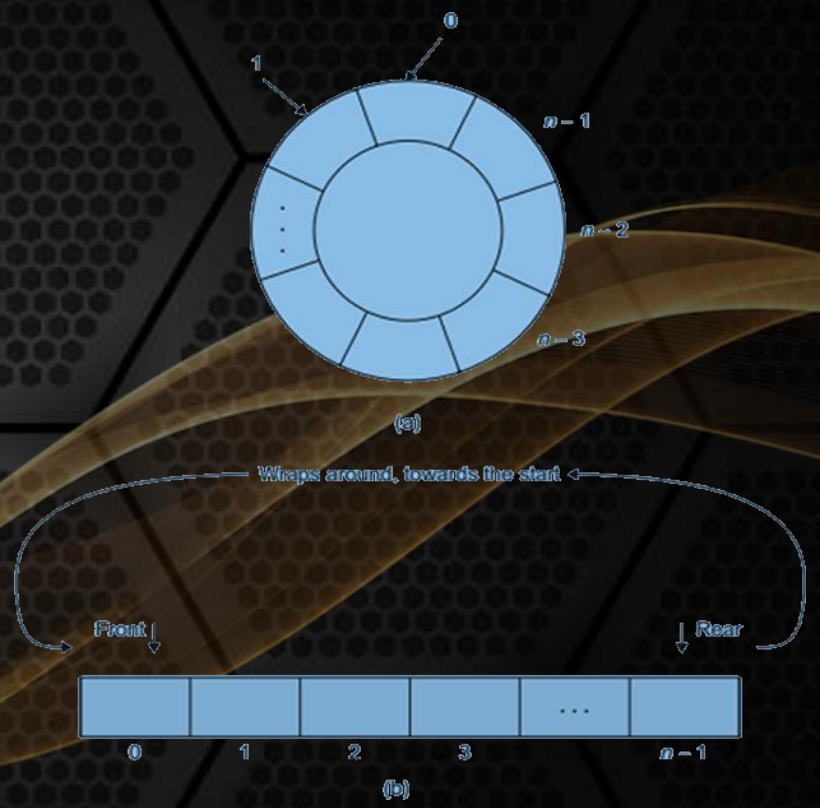
Supongamos que la cola Q es de tamaño n. Ahora, si seguimos añadiendo elementos en la cola, podemos llegar a la ubicación n - 1. Si no es circular, no se pueden agregar más elementos aunque haya ubicaciones vacías en la parte delantera de la array. En cambio, si hay ubicaciones vacías en la parte delantera, usando una cola circular podemos agregar elementos en esa ubicación en lugar de señalar un error ya que la cola está llena o está cambiando los datos. Las ranuras vacías se llenarán con nuevos elementos entrantes aunque Trasero = n - 1. De ahí que la cola circular nos permita seguir añadiendo elementos aunque hayamos llegado al final de la array. Se dice que la cola está llena solo cuando hay n elementos en la cola.



Queue (Colas) | Circular Queue

Consideremos la cola Q que es de tamaño n. Ya hemos estudiado las operaciones en colas lineales utilizando array. También estudiamos sus funciones correspondientes en el lenguaje C++. Veamos si se pueden utilizar las mismas funciones para colas circulares. En una cola circular, cuando la parte trasera es n - 1 y se va a agregar un nuevo elemento, la parte trasera debe establecerse en 0.

Inicialmente, tanto la parte delantera como la trasera se establecen en -1. El valor del frente siempre será uno menos que el del frente real. Las funciones para agregar y eliminar elementos se reescriben con algunas modificaciones en el código del programa



Queue (Colas) | Circular Queue Example

La implementación de una cola circular utilizando una array se proporciona en el código de programa. Veamos su trabajo con un ejemplo. Considere max = 5 e inicialmente, Delantero = Trasero = 0.

Para comprobar las condiciones Queue_Full y Queue_Empty, debemos comprobar si los valores de Delante y Detrás son iguales. En los lenguajes de programación C/C++, el índice de array varía de 0 a n - 1, de modo que una ubicación de la cola circular siempre permanece sin usar. Tal no es el caso en lenguajes como el Pascal. Por lo tanto, en una cola circular que utiliza matrices en C/C++, podemos almacenar n - 1 elementos, donde n se declara como el tamaño de la array. Por lo tanto, para almacenar n elementos, debemos declarar la array de tamaño n + 1.

0	1	2	3	4	Front	Rear	Action
					0	0	`Q_Empty`
	11				0	1	Insert 11
	11	12			0	2	Insert 12
	11	12	13		0	3	Insert 13
	11	12	13	14	0	4	Insert 14
	11	12	13	14	0		Insert 15 Can't insert 15 as `Q_Full` since `Rear` = (4 + 1)%5 = 0 which is equal to `Front`
	–	12	13	14	1	4	Delete
		–	13	14	2	4	Delete
			–	14	3	4	Delete
				–	4	4	Delete Can't delete as `Front = Rear` makes `Q_Empty`

Ventajas de usar colas circulares

Los siguientes son los méritos de usar colas circulares:

1. Al usar colas circulares, se evita el cambio de datos ya que la parte delantera y trasera se modifican mediante el uso de la función mod(). La operación mod()envuelve la cola hasta su inicio.
2. Si el número de elementos a almacenar en la cola es fijo (es decir, si el tamaño de la cola es específico), la cola circular es ventajosa.
3. Muchas aplicaciones prácticas, como la cola de impresoras, la cola de prioridades y las simulaciones, utilizan la cola circular.

Queue (Colas) | Circular Queue Example

```
if( rear == MAX -1 )
 rear = 0;
else
 rear = rear + 1;
```

The short-hand equivalent representation may be

```
rear = ( rear + 1) % MAX;
```

```
#include<iostream>
using namespace std;

class Cqueue
{
 private:
 int Rear, Front;
 int Queue[50];
 int Max;
 int Size;

 public:

 Cqueue() {Size = 0; Max = 50;
Rear = Front = -1;}
 int Empty();
 int Full();
 void Add(int Element);
 int Delete();
 int getFront();
};

int Cqueue :: Empty()
{
 if(Front == Rear)
 return 1;
 else
 return 0;
}
```

```
int Cqueue :: Full()
{
 if(Rear == Front)
 return 1;
 else
 return 0;
}

void Cqueue :: Add(int Element)
{
 if(!Full())
 Rear = (Rear + 1) % Max;
 Queue[Rear] = Element;
 Size++;
}

int Cqueue :: Delete()
{
 if(!Empty())
 Front = (Front + 1) % Max;
 Size--;
 return(Queue[Front]);
}

int Cqueue :: getFront()
{
 int Temp;
 if(!Empty())
 Temp = (Front + 1) % Max;
 return(Queue[Temp]);
}
```

```
void main(void)
{
 Cqueue Q;
 Q.Add(11);
 Q.Add(12);
 Q.Add(13);
 cout << Q.Delete() << endl;
 Q.Add(14);
 cout << Q.Delete() << endl;
 cout << Q.Delete() << endl;
 cout << Q.Delete() << endl;
 cout << Q.Delete() << endl;
 Q.Add(15);
 Q.Add(16);
 cout << Q.Delete() << endl;
}
```



CQueue.cpp

Colas Multiqueues | Introducción.

MultiQueue

Queue (Colas) | Multiqueues

Si se requiere implementar más número de colas, se requiere una estructura de datos eficiente para manejar múltiples colas. Es posible utilizar todos los espacios disponibles en una sola array. Cuando más de dos colas, digamos n, se representan secuencialmente, podemos dividir la memoria disponible A[size] en n segmentos y asignar estos segmentos a n colas, una a cada una. Para cada cola i usaremos Front[i] y Rear[i]. Usaremos la condición Front[i] = Rear[i] si y solo si la i ésima cola está vacía, y la condición Rear[i] = Front[i] si y solo si la i ésima cola está llena. Si queremos cinco colas, entonces podemos dividir la array A[100] en partes iguales de 20 e inicializar delante y detrás para cada cola, es decir, Front[0] = Rear[0] = 0 y Front[1] = Rear[1] = 20, y así sucesivamente para otras colas (Fig).



Fig. 5.3 A multi-queue

After adding elements 5 and 8 in the second queue, the resultant queue will be as in Fig. 5.4.

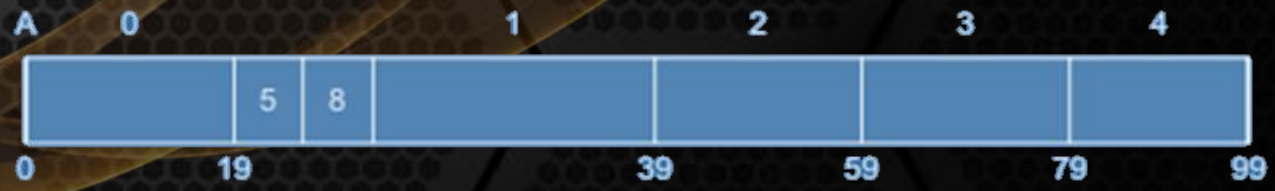


Fig. 5.4 Queue in Fig. 5.3 after addition of elements

Colas Doble Extremo| Introducción.

deQue

Queue (Colas) | Deque

La palabra **deque** es una forma corta de cola de doble extremo. Se pronuncia como 'deck'. Deque define una estructura de datos en la que los elementos se pueden agregar o eliminar en el extremo frontal o en el extremo posterior, pero no se pueden realizar cambios en ninguna otra parte de la lista. Por lo tanto, deque es una generalización tanto de una pila como de una cola. Es compatible con capacidades similares a pilas y colas. Es un contenedor secuencial que está optimizado para un acceso rápido basado en índices y una inserción eficiente en cualquiera de sus extremos. Deque se puede implementar como un deque continuo o como un deque vinculado. La figura muestra la representación de un deque.

El deque ADT combina las características de pilas y colas. Al igual que las pilas y colas, un deque permite acceder a los elementos solo en los extremos. Sin embargo, un deque permite agregar y eliminar elementos de cada extremo. Podemos referirnos a las operaciones soportadas por el deque como EnqueueFront, EnqueueRear, DequeueFront y DequeueRear. Cuando completamos una descripción formal del deque y luego lo implementamos utilizando una implementación dinámica y vinculada, podemos usarlo para implementar tanto pilas como colas, logrando así una reutilización significativa del código.

Las siguientes son las cuatro operaciones asociadas con deque:

1. EnqueueFront(): agrega elementos en el extremo frontal de la cola
2. EnqueueRear(): agrega elementos en la parte trasera de la cola
3. DequeueFront(): elimina elementos del front-end de la cola
4. DequeueRear(): elimina elementos del extremo posterior de la cola

Para la implementación de la pila mediante deque, EnqueueFront y DequeueFront se utilizan como funciones push y pop, respectivamente.

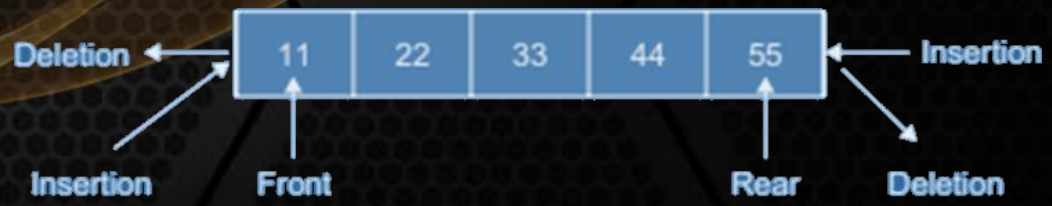


Fig. 5.5 Representation of a deque

Queue (Colas) | Deque

Las aplicaciones de deque

Deque son útiles donde los datos a almacenar tienen que ser ordenados, se necesita un almacenamiento compacto, y la recuperación de los elementos de datos tiene que ser más rápida.

Variaciones de deque

Podemos tener dos variaciones de un deque: el deque restringido de entrada y el deque restringido de salida. El deque restringido de salida permite eliminaciones de un solo extremo y el deque restringido de entrada permite inserciones solo en un extremo.

Las funciones para operar una deque restringida a la salida podrían ser las siguientes:

DequeueFront()(o DequeueRight()), EnqueueFront() y EnqueueRear()

Las funciones para operar una deque restringida a entrada son las siguientes:

DequeueFront(), DequeueRight() y EnqueueFront()(o EnqueueRear())

Colas Prioritarias | Introducción.

Priority Queue

Queue (Colas) | Priority Queue

Una cola de prioridad Priority Queue es una colección de un número finito de elementos priorizados. Las colas de prioridad son aquellas en las que podemos insertar o eliminar elementos de cualquier posición en función de algún orden fundamental de los elementos.

Los elementos se pueden insertar en cualquier orden en una cola de prioridad, pero cuando un elemento se elimina de la cola de prioridad, siempre es el que tiene la prioridad más alta. En otras palabras***, una cola de prioridad es una colección de elementos donde los elementos se almacenan de acuerdo con sus niveles de prioridad.***

El orden en que deben eliminarse los elementos se decide por la prioridad del elemento.

Se aplican las siguientes reglas para mantener una cola de prioridad:

1. El elemento con una prioridad más alta se procesa antes que cualquier elemento de menor prioridad.
2. Si hubiera elementos con la misma prioridad, entonces el elemento agregado primero en la cola se procesaría primero.

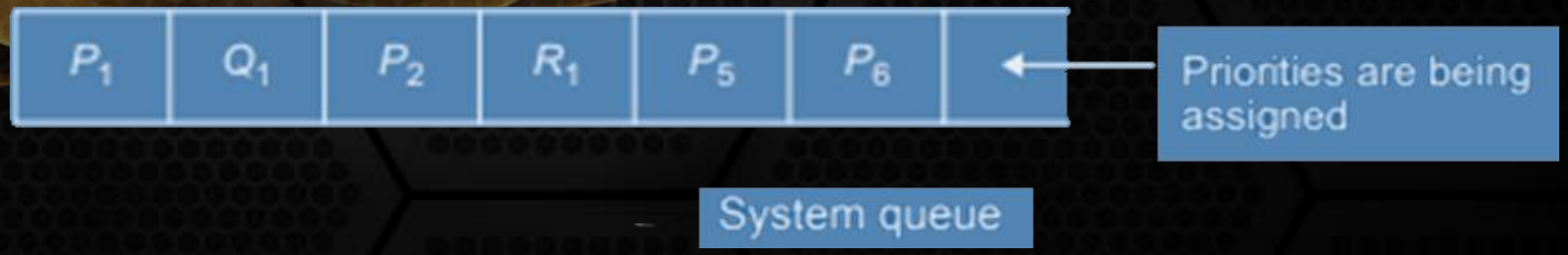
¿Cómo implementar la cola de prioridad?
La cola de prioridad se puede implementar utilizando las siguientes estructuras de datos:
- Matrices
- Lista vinculada
- Estructura de datos del montón
- Árbol de búsqueda binaria

Queue (Colas) | Priority Queue

Las colas de prioridad se utilizan para implementar la programación de trabajos por parte del sistema operativo, donde los trabajos con mayor prioridad deben procesarse primero. Otra aplicación de las colas de prioridad es en sistemas de simulación donde la prioridad corresponde a los tiempos de los eventos.

Los siguientes son algunos ejemplos de una cola de prioridad:

1. Una lista de pacientes en una sala de emergencias; a cada paciente se le puede dar una clasificación que depende de la gravedad de la enfermedad del paciente.
2. Una lista de los trabajos realizados por un sistema operativo multitarea; a cada trabajo en segundo plano se le da un nivel de prioridad. Supongamos que en un sistema informático, a los trabajos se les asignan tres prioridades, a saber, P, Q, R como primera, segunda y tercera, respectivamente. De acuerdo con la prioridad del trabajo, se inserta al final de los otros trabajos que tienen la misma prioridad. Considere la cola de prioridad dada en la Fig.



Queue (Colas) | Array Implementation of Priority Queue

Al igual que las pilas y las colas, incluso una cola de prioridad se puede representar mediante una array. Sin embargo, si se utiliza cualquier array para almacenar elementos de una cola de prioridad, entonces la inserción de elementos en la cola sería fácil, pero la eliminación de elementos sería difícil. Esto se debe a que al insertar elementos en la cola de prioridad, no se insertan en un orden. Como resultado, eliminar un elemento con la prioridad más alta requeriría examinar toda la array para buscar dicho elemento. Además, un elemento de una cola solo se puede eliminar del front-end. No hay una solución satisfactoria a este problema. Sin embargo, sería más eficiente si almacenamos los elementos en una cola de prioridad. Cada elemento de una array puede tener la siguiente estructura

```
typedef struct
{
 int Data;
 int priority;
 int order;
}
```

donde `priority` representa la prioridad del elemento y orden representa el orden en que el elemento se ha agregado a la cola.

Queue (Colas) | Array Implementation of Priority Queue

Al igual que las pilas y las colas, incluso una cola de prioridad se puede representar mediante una array. Sin embargo, si se utiliza cualquier array para almacenar elementos de una cola de prioridad, entonces la inserción de elementos en la cola sería fácil, pero la eliminación de elementos sería difícil. Esto se debe a que al insertar elementos en la cola de prioridad, no se insertan en un orden. Como resultado, eliminar un elemento con la prioridad más alta requeriría examinar toda la array para buscar dicho elemento. Además, un elemento de una cola solo se puede eliminar del front-end. No hay una solución satisfactoria a este problema. Sin embargo, sería más eficiente si almacenamos los elementos en una cola de prioridad. Cada elemento de una array puede tener la siguiente estructura

```
typedef struct
{
 int Data;
 int priority;
 int order;
}
```

donde `priority` representa la prioridad del elemento y orden representa el orden en que el elemento se ha agregado a la cola.

Queue (Colas) | Array Implementation of Priority Queue

```cpp
// C++ program to implement Priority Queue
// using Arrays
#include <bits/stdc++.h>
using namespace std;

// Structure for the elements in the
// priority queue
struct item {
    int value;
    int priority;
};

// Store the element of a priority queue
item pr[100000];

// Pointer to the last index
int size1 = -1;

// Function to insert a new element
// into priority queue
void enqueue(int value, int priority)
{
    // Increase the size
    size1++;

    // Insert the element
    pr[size1].value = value;
    pr[size1].priority = priority;
}
```

```cpp
// Function to check the top element
int peek()
{
    int highestPriority = INT_MIN;
    int ind = -1;

    // Check for the element with
    // highest priority
    for (int i = 0; i <= size1; i++) {

        // If priority is same choose the element with the
        // highest value
        if (highestPriority == pr[i].priority && ind > -1
            && pr[ind].value < pr[i].value) {
            highestPriority = pr[i].priority;
            ind = i;
        }
        else if (highestPriority < pr[i].priority) {
            highestPriority = pr[i].priority;
            ind = i;
        }
    }

    // Return position of the element
    return ind;
}
```

```cpp
// Function to remove the element with
// the highest priority
void dequeue()
{
    // Find the position of the element
    // with highest priority
    int ind = peek();

    // Shift the element one index before
    // from the position of the element
    // with highest priority is found
    for (int i = ind; i < size1; i++) {
        pr[i] = pr[i + 1];
    }

    // Decrease the size of the
    // priority queue by one
    size1--;
}
```

Queue (Colas) | Array Implementation of Priority Queue

```cpp
// C++ program to implement Priority Queue
// using Arrays
#include <bits/stdc++.h>
using namespace std;

// Structure for the elements in the
// priority queue
struct item {
    int value;
    int priority;
};

// Store the element of a priority queue
item pr[100000];

// Pointer to the last index
int size1 = -1;

// Function to insert a new element
// into priority queue
void enqueue(int value, int priority)
{}

// Function to check the top element
int peek()
{}

// Function to remove the element with
// the highest priority
void dequeue()
{}
```

```cpp
// Driver Code
int main()
{
    // Function Call to insert elements as per the priority
    enqueue(10, 2);
    enqueue(14, 4);
    enqueue(16, 4);
    enqueue(12, 3);

    // Stores the top element at the moment
    int ind = peek();

    cout << pr[ind].value << endl;

    // Dequeue the top element
   dequeue();

    // Check the top element
    ind = peek();
    cout << pr[ind].value << endl;

    // Dequeue the top element
    dequeue();

    // Check the top element
    ind = peek();
    cout << pr[ind].value << endl;

    return 0;
}
```



Queue_Priority.cpp

Queue en la STL| Introducción.

Queue / Deque in the Standard Template Library

Queue (Colas) | QUEUES USING TEMPLATE

Las colas son un tipo de adaptadores de contenedores que operan en un tipo de disposición de primero en entrar, primero en salir **(FIFO)**. Los elementos se insertan en la parte posterior (extremo) y se eliminan de la parte delantera. Las colas utilizan un objeto encapsulado de deque o list (clase de contenedor secuencial) como contenedor subyacente, proporcionando un conjunto específico de funciones miembro para acceder a sus elementos.

A continuación se muestra un ejemplo para demostrar la cola y sus diversos métodos.

La complejidad temporal y la definición de las siguientes funciones son las siguientes:

cola::vacío() O(1)
cola::size() O(1)
cola::emplace() O(1)
cola::front() O(1)
cola::back() O(1)
cola::p ush(g) O(1)
cola::p op() O(1)

Método	Definición
cola::vacío()	Devuelve si la cola está vacía. Devuelve true si la cola está vacía, de lo contrario devuelve false.
cola::size()	Devuelve el tamaño de la cola.
cola::swap()	Intercambie el contenido de dos colas, pero las colas deben ser del mismo tipo de datos, aunque los tamaños pueden diferir.
cola::emplace()	Inserte un nuevo elemento en el contenedor de colas, el nuevo elemento se agrega al final de la cola.
cola::front()	Devuelve una referencia al primer elemento de la cola.
cola::back()	Devuelve una referencia al último elemento de la cola.
cola::p ush(g)	Añade el elemento 'g' al final de la cola.
cola::p op()	Elimina el primer elemento de la cola.

Queue (Colas) | QUEUES USING TEMPLATE

```cpp
// CPP code to illustrate Queue in
// Standard Template Library (STL)
#include <iostream>
#include <queue>

using namespace std;

// Print the queue
void showq(queue<int> gq)
{
    queue<int> g = gq;
    while (!g.empty()) {
        cout << '\t' << g.front();
        g.pop();
    }
    cout << '\n';
}

// Driver Code
int main()
{
    queue<int> gquiz;
    gquiz.push(10);
    gquiz.push(20);
    gquiz.push(30);

    cout << "The queue gquiz is : ";
    showq(gquiz);

    cout << "\ngquiz.size() : " << gquiz.size();
    cout << "\ngquiz.front() : " << gquiz.front();
    cout << "\ngquiz.back() : " << gquiz.back();

    cout << "\ngquiz.pop() : ";
    gquiz.pop();
    showq(gquiz);

    return 0;
}
```

Salida

```
The queue gquiz is :    10    20    30

gquiz.size() : 3
gquiz.front() : 10
gquiz.back() : 30
gquiz.pop() :    20    30
```



Queue_STL.cpp

deQue in STL| Introducción.

Deque in STL

Deque (Colas) | DEQUE USING TEMPLATE

Las colas de doble extremo son contenedores de secuencia con la característica de expansión y contracción en ambos extremos. Son similares a los vectores, pero son más eficientes en caso de inserción y eliminación de elementos. A diferencia de los vectores, la asignación de almacenamiento contiguo puede no estar garantizada.
Las colas de doble extremo son básicamente una implementación de la estructura de datos de cola de doble extremo. Una estructura de datos de cola permite la inserción solo al final y la eliminación desde el frente. Esto es como una cola en la vida real, en la que las personas se eliminan del frente y se agregan en la parte posterior. Las colas de doble extremo son un caso especial de colas donde las operaciones de inserción y eliminación son posibles en ambos extremos.
Las funciones para deque son las mismas que las vectoriales, con una adición de operaciones push y pop tanto para delante como para atrás.

Las complejidades de tiempo para realizar diversas operaciones en deques son-

Acceso a elementos- O(1)
Inserción o extracción de elementos- O(N)
Inserción o extracción de elementos al inicio o al final- O(1)

Deque (Colas) | DEQUE USING TEMPLATE

Método	Definición
deque::insert()	Inserta un elemento. Y devuelve un iterador que apunta al primero de los elementos recién insertados.
deque::rbegin()	Devuelve un iterador inverso que apunta al último elemento del deque (es decir, su inicio inverso).
deque::rend()	Devuelve un iterador inverso que apunta a la posición anterior al principio de la deque (que se considera su extremo inverso).
deque::cbegin()	Devuelve un iterador constante que apunta al primer elemento del contenedor, es decir, el iterador no se puede usar para modificar, solo atravesar el deque.
deque::max_size()	Devuelve el número máximo de elementos que puede contener un contenedor de deque.
deque::asignar()	Asigne valores al mismo o diferente contenedor de deque.
deque::resize()	Función que cambia el tamaño del deque.
deque::p ush_front()	Se utiliza para empujar elementos en un deque desde el frente.
deque::p ush_back()	Esta función se utiliza para empujar elementos en un deque desde la parte posterior.
deque::p op_front() y deque::p op_back()	pop_front() se utiliza para hacer estallar o eliminar elementos de un deque desde el frente. pop_back() se utiliza para hacer estallar o eliminar elementos de un deque de la parte posterior.
deque::front() y deque::back()	la función front() se utiliza para hacer referencia al primer elemento del contenedor deque. La función back() se utiliza para hacer referencia al último elemento del contenedor de deque.
deque::clear() y deque::erase()	La función clear() se utiliza para eliminar todos los elementos del contenedor deque, haciendo así su tamaño 0. La función erase() se utiliza para eliminar elementos de un contenedor de la posición o rango especificados.
deque::empty() y deque::size()	la función empty() se utiliza para comprobar si el contenedor deque está vacío o no. La función size() se utiliza para devolver el tamaño del contenedor deque o el número de elementos en el contenedor deque.
deque::operator= y deque::operator[]	operator= operator se utiliza para asignar nuevos contenidos al contenedor reemplazando el contenido existente. operador[] operador se utiliza para hacer referencia al elemento presente en la posición dada dentro del operador.
deque::at() y deque::swap()	se utiliza la función at() como referencia al elemento presente en la posición dada como parámetro de la función. La función swap() se utiliza para intercambiar el contenido de un deque con otro deque del mismo tipo y tamaño.
deque::begin() y deque::end	La función begin() se utiliza para devolver un iterador que apunta al primer elemento del contenedor deque. La función end() se utiliza para devolver un iterador que apunta al último elemento del contenedor de deque.
deque::emplace_front() y deque::emplace_back()	emplace_front función() se utiliza para insertar un nuevo elemento en el contenedor de deque. El nuevo elemento se añade al principio del deque. emplace_back() se utiliza para insertar un nuevo elemento en el contenedor de deque. El nuevo elemento se añade al final del deque.

Deque (Colas) | DEQUE USING TEMPLATE

Las complejidades de tiempo para realizar diversas operaciones en deques son-

Acceso a elementos- O(1)
Inserción o extracción de elementos- O(N)
Inserción o extracción de elementos al inicio o al final- O(1)

Salida
The deque gquiz is : 15 20 10 30

gquiz.size() : 4
gquiz.max_size() : 4611686018427387903
gquiz.at(2) : 10
gquiz.front() : 15
gquiz.back() : 30
gquiz.pop_front() : 20 10 30

gquiz.pop_back() : 20 10

```cpp
// CPP Program to implement Deque in STL
#include <deque>
#include <iostream>

using namespace std;

void showdq(deque<int> g)
{
    deque<int>::iterator it;
    for (it = g.begin(); it != g.end(); ++it)
        cout << '\t' << *it;
    cout << '\n';
}

int main()
{
    deque<int> gquiz;
    gquiz.push_back(10);
    gquiz.push_front(20);
    gquiz.push_back(30);
    gquiz.push_front(15);
    cout << "The deque gquiz is : ";
    showdq(gquiz);

    cout << "\ngquiz.size() : " << gquiz.size();
    cout << "\ngquiz.max_size() : " << gquiz.max_size();

    cout << "\ngquiz.at(2) : " << gquiz.at(2);
    cout << "\ngquiz.front() : " << gquiz.front();
    cout << "\ngquiz.back() : " << gquiz.back();

    cout << "\ngquiz.pop_front() : ";
    gquiz.pop_front();
    showdq(gquiz);

    cout << "\ngquiz.pop_back() : ";
    gquiz.pop_back();
    showdq(gquiz);

    return 0;
}
```

Dque_STL.cpp

Queue| Resumen.

Queue (Colas) | Resumen

• Una cola es una lista ordenada donde todas las inserciones se realizan en un extremo llamado parte posterior y las eliminaciones en otro extremo llamado frente. Estos límites garantizan que los datos se procesen en la secuencia en la que se ingresan.
En resumen, una cola es una estructura fifo o last in last out (LILO).

• Una cola es una estructura de datos lineal, ya que se puede implementar con la ayuda de matrices (utilizando la asignación de memoria estática) o listas vinculadas (utilizando la asignación de memoria dinámica). Una array no es una estructura de datos adecuada para la inserción y eliminación frecuente de elementos de datos. Además, utiliza la asignación de memoria estática para poder almacenar un número de elementos. Por lo tanto, la implementación de arreglos de discos no es adecuada para inserciones y eliminaciones frecuentes.

• Estos inconvenientes se pueden evitar implementando la cola utilizando una array circular. En una cola circular, a medida que vamos añadiendo elementos a la cola y llegando al final de la array, el siguiente elemento se almacena en la primera ranura de la array si está libre.

• Hay variaciones de colas como cola circular, colas múltiples y deque. Deque defi nes una estructura de datos donde los elementos se pueden agregar o eliminar en el extremo frontal o en el extremo posterior, pero no se pueden realizar cambios en otra parte de la lista.

• Las colas se utilizan en muchas aplicaciones, como simulación, cola de prioridad, cola de trabajos, etc. Las colas de prioridad son aquellas en las que podemos insertar o eliminar elementos de cualquier posición en función de algún orden fundamental con respecto a las prioridades de los elementos.

Muchas Gracias.

Nos vemos en Recursión

